

Name: Logan Meyers
WSU ID: 011881121
Date: 09/30/2026
Class: CPTS 360
Assignment: PA1 Report

PA1 Report - Summary of Implementation

To start, I'd like to say this was all-in-all pretty easy. At first it seemed complicated but once you boil it down to the important pieces and the subproblems, it gets a lot easier- like with most problems. My strategy was the following:

1. Read the `pa1_documentation.pdf` to get an idea of the assignment and requirements
2. Practice cache concepts to make sure I have a solid understanding of how caches work, so I know how I need to implement it in code
3. Implement it in code

Overall, that was my strategy. Here's how I did each part:

Reading the documentation was straightforward. The instructions were clear, the given code was tremendously useful (I've never used `getopt` before, the sample code was really nice), and the test cases were very nice to have.

Next was cache concepts. I used a mix of the course lecture materials, in-class exercises, and Claude-generated examples for me to try out on my own, run through the instructions, and make sure I understood how splitting the address apart into the set, tag, and offset parts worked. I also did this to make sure I know how each case went; e.g. when hits happen, when misses happen, when evictions happen, what cases loads can have, stores can have, and modifies can have. This was actually super helpful because I figured out pretty early on that I could condense the load (L) and store (S) cases into one in the simulator, since the hits, misses, and evictions work the same for both since we're not actually moving any memory.

Speaking of not moving memory, having our simulator completely ignore moving memory and handling bytes of memory and offsets and all of that made this much easier. After practicing the concepts, I knew the natural choice was to use a dynamically-allocated 2D array in C for the cache, with rows being sets and columns being the lines. (Even though it ended up being a flat array, it still worked pretty much the same). I used a struct called `Line` as the type of data for the array, which held info about the line's validity, tag, and least-recently-used (LRU) number.

Then it was all down to actually implementing it. Working from the given starter code, I started by setting up most of the variables I would need in the `main()` function like counters, cache specs (`s`, `E`, `b`), file pointers, and some other things. I then put in the `getopt` loop, set up the switch case for that, put in a check for invalid parameters, and made sure that worked by printing out the variables. All good!

Next was file management. I've never been great at string manipulation in C, so it took some struggling and much googling (basic usage of things like `strdup`, `basename`, `strchr`, etc). Eventually I set

up some simple file open functions for the infile and outfile, with hardcoded paths, mostly. It could be more modular, but for this hardcoding the /traces/output/ directory should be fine. Later on I added a couple more functions related to file management (output trace detail writing) as I neared the end.

Next, and most importantly, was the actual cache simulation. I made a function to create the dynamic memory for a cache with the given parameters. I created some enums (and a helper function to convert them to strings for outputting) for access types and results. Then I split the flow into a general `main` → `cacheSimLoop` → `accessSet` flow. Main calls the sim loop, whose job is to read the trace, extract the info, then access the cache and write out the results and update counters. S and L cases were the same: one access, write one result. M acted as a L then S, so two access and write two results in one go.

The `accessSet` function had 3 steps, all looping through the lines to find the best case:

1. Try to find a valid line with matching tag; update LRU
2. Try to find an empty (nonvalid) line; set tag, valid, and set LRU
3. Find the LRU line and replace the tag and update LRU number

Overall, that's my implementation! This was actually pretty fun to make and I enjoyed the process and the satisfying result.